EF Core vs Dapper: Enterprise Application Discussion

Question 1: Evaluating EF Core Overhead

User: as a c# developer, I write enterprise applications that connect to Mssql database servers. currently I use EF Core and only use stored procedures, no LINq or Lamda or entities. we also never generate database objects from code. so all those features are not needed. I am concerned of the bloat and overhead I am adding by continuing to use ef core. using your best judgment as a full stack expert, provide a better option and the reasoning behind your decision.

Claude: Given your constraints and concerns, **I'd recommend switching to Dapper**.

Here's my reasoning:

Why Dapper fits your scenario perfectly:

Dapper is a micro-ORM that sits right in the sweet spot for your use case. You're essentially using EF Core as an expensive stored procedure executor, which is like using a bulldozer to hammer a nail. Dapper gives you direct SQL execution with minimal overhead while still providing the convenience of mapping results to C# objects.

Key advantages for your situation:

Performance and overhead: Dapper adds virtually no overhead beyond raw ADO.NET. It's just a thin mapping layer. EF Core includes change tracking, query translation, entity graphs, migrations, and tons of features you're explicitly not using. You're paying that cost in memory and startup time for nothing.

Alignment with your architecture: Since you're committed to stored procedures and database-first design, Dapper embraces this rather than fighting it. EF Core is fundamentally designed around entities and code-first/database-first generation, so you're swimming upstream.

Simplicity: Your codebase will be more explicit and easier to understand. With Dapper, what you see is what executes. No hidden queries, no surprise N+1 problems, no dependency injection complexity you don't need.

Code comparison:

Your current EF Core approach probably looks like:

```
var results = context.Database.SqlQueryRaw<MyModel>("EXEC GetCustomers @param",
param).ToList();
```

With Dapper:

```
var results = connection.Query<MyModel>("EXEC GetCustomers @param", new { param
}).ToList();
```

Functionally similar, but Dapper is purpose-built for this.

The one caveat: If you're using EF Core's connection resiliency features (automatic retries, connection pooling management), you'll need to implement those yourself or use something like Polly. But that's typically straightforward for enterprise applications.

For your specific use case—stored procedures only, no code generation, enterprise SQL Server applications—Dapper is the pragmatic choice. You'll reduce dependencies, improve performance, and have code that more honestly represents what it's doing.

Question 2: ADO.NET vs Dapper Comparison

User: ok, but since you mentioned ado.net, compare that to Dapper and would ado.net support connection pooling

Claude: Good question. Let me compare raw ADO.NET to Dapper directly.

ADO.NET vs Dapper:

ADO.NET absolutely supports connection pooling—it's built into the SqlConnection class and enabled by default. When you open/close connections properly (using `using` statements), the connection pool manages everything automatically. This is a core .NET feature, not an EF Core thing.

The real comparison:

Raw ADO.NET gives you the absolute lowest overhead possible. You're working directly with `SqlCommand`, `SqlParameter`, and `SqlDataReader`. Maximum control, zero abstraction.

Dapper is literally just ADO.NET with automatic object mapping. It uses ADO.NET under the hood—it's essentially extension methods on `IDbConnection` that handle the tedious boilerplate of mapping `SqlDataReader` rows to your objects.

Code comparison:

ADO.NET:

```
using (var connection = new SqlConnection(connectionString))
using (var command = new SqlCommand("EXEC GetCustomer @Id", connection))
{
    command.Parameters.AddWithValue("@Id", customerId);
    connection.Open();
    using (var reader = command.ExecuteReader())
    {
        while (reader.Read())
        {
            var customer = new Customer
            {
                Id = reader.GetInt32(0),
                Name = reader.GetString(1),
                Email = reader.IsDBNull(2) ? null : reader.GetString(2)
                // ... map each field manually
            };
        }
    }
}
```

Dapper:

```
using (var connection = new SqlConnection(connectionString))
{
    var customer = connection.QueryFirstOrDefault<Customer>(
        "EXEC GetCustomer @Id",
        new { Id = customerId }
    );
}
```

My recommendation:

For enterprise applications, **I'd still choose Dapper** over raw ADO.NET for these reasons:

Developer productivity: The mapping boilerplate in ADO.NET is tedious and error-prone.

You're constantly dealing with ordinal positions, null checking, and type casting. Dapper eliminates this while adding negligible overhead.

Maintainability: When stored procedures change (add/remove columns), Dapper code often just works if your model matches. ADO.NET code requires manual updates to every field mapping.

Performance is essentially identical: Dapper's overhead is measured in microseconds. Unless you're processing millions of rows per second, you won't notice the difference.

Still uses connection pooling: Dapper uses ADO.NET connections directly, so you get all the same pooling benefits.

When to use raw ADO.NET instead:

- Extremely high-performance scenarios where every microsecond counts
- Streaming large datasets where you need fine-grained control over the reader
- Very simple operations where you're only reading a single value

For typical enterprise CRUD operations with stored procedures, Dapper hits the perfect balance of performance and productivity. You're not paying for features you don't need (like EF Core), but you're also not drowning in mapping code (like raw ADO.NET).

Question 3: Measuring Performance Improvements

User: ok, Im sold on Dapper. I will be asked to show the improvements that you have stated. what is the best way for me to measure the before and after of converting from EF to Dapper?

Claude: Great question. You need concrete, defensible metrics. Here's how to measure the improvements:

1. Benchmark with BenchmarkDotNet

This is the gold standard for .NET performance testing. It handles warmup, statistical analysis, and eliminates measurement noise.

```
[MemoryDiagnoser]
[SimpleJob(RuntimeMoniker.Net80, baseline: true)]
public class EfVsDapperBenchmark
{
    private string _connectionString;
    private YourDbContext _context;

    [GlobalSetup]
    public void Setup()
    {
        _connectionString = "your-connection-string";
        _context = new YourDbContext();
    }

    [Benchmark(Baseline = true)]
    public List<Customer> EfCore_StoredProc()
    {
        return _context.Database
            .SqlQueryRaw<Customer>("EXEC GetCustomers")
            .ToList();
    }

    [Benchmark]
    public List<Customer> Dapper_StoredProc()
    {
        using var connection = new SqlConnection(_connectionString);
        return connection.Query<Customer>("EXEC GetCustomers").ToList();
    }
}
```

Install: **`dotnet add package BenchmarkDotNet`**

Run it and you'll get output showing:

- Execution time (mean, median, std dev)
- Memory allocations (Gen0, Gen1, allocated bytes)
- Statistical comparison with confidence intervals

2. Application startup time

Measure cold start performance:

```
var sw = Stopwatch.StartNew();
// EF Core initialization
var context = new YourDbContext();
var dummy = context.Database.CanConnect(); // Force initialization
sw.Stop();
Console.WriteLine($"EF Core startup: {sw.ElapsedMilliseconds}ms");
```

vs

```
var sw = Stopwatch.StartNew();
// Dapper initialization (basically none)
using var connection = new SqlConnection(connectionString);
connection.Open();
sw.Stop();
Console.WriteLine($"Dapper startup: {sw.ElapsedMilliseconds}ms");
```

3. Deployment package size

Compare the published output:

```
# Before (EF Core)
dotnet publish -c Release
# Check bin/Release/net8.0/publish folder size

# After (Dapper)
dotnet publish -c Release
# Compare the difference
```

You'll see significant reduction in DLL count and total size.

4. Memory profiling in production-like scenarios

Use a simple load test to measure sustained memory usage:

```
// Run 1000 operations and measure
var memoryBefore = GC.GetTotalMemory(true);

for (int i = 0; i < 1000; i++)
{
    // Your typical operation
    var results = GetCustomers();
}

var memoryAfter = GC.GetTotalMemory(true);
Console.WriteLine($"Memory delta: {(memoryAfter - memoryBefore) / 1024 / 1024}MB");
```

5. Real-world endpoint testing

If you have APIs, use a load testing tool like k6, JMeter, or even just `wrk`:

```
# Test your API endpoint before conversion
wrk -t4 -c100 -d30s http://localhost:5000/api/customers

# Note: requests/sec, latency p50, p99
# Test after conversion and compare
```

What to present to stakeholders:

Create a simple comparison table:

Metric	EF Core	Dapper	Improvement
Avg execution time	X ms	Y ms	Z% faster
Memory per operation	X KB	Y KB	Z% reduction
Startup time	X ms	Y ms	Z% faster
Published size	X MB	Y MB	Z MB smaller
LOC for typical operation	X lines	Y lines	Simpler

My recommendation for your presentation:

Focus on **3 key metrics** that matter to enterprise apps:

1. **Memory efficiency** (lower GC pressure = better scalability)
2. **Startup time** (faster deployments, container starts)
3. **Package size** (smaller deployments, less attack surface)

The execution time difference might be small for single operations, but the memory and deployment improvements will be dramatic and easy to demonstrate. Run BenchmarkDotNet on 2-3 of your most common stored procedure calls, and you'll have compelling, statistically valid data.

Summary

This conversation covered:

1. Why Dapper is a better fit than EF Core for stored-procedure-only enterprise applications
2. How Dapper compares to raw ADO.NET (with connection pooling confirmation)
3. Concrete methods to measure and demonstrate performance improvements when migrating from EF Core to Dapper

Key takeaway: For your use case, Dapper provides the optimal balance of performance, simplicity, and developer productivity without the overhead of unused EF Core features.